

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

Large-Scale Traffic Pattern Mining and Detection-Consistency Analysis

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Big Data Processing

Contents

1 Data preparation and access experiment 2

2 Market baskets and time-window sensitivity 4

3 Frequent triples and algorithm choice 7

4 Camera and contractor miss heuristics 11

5 MinHash/LSH route-set similarity 13

6 Engineering interpretation and preserved results 16

1 Data preparation and access experiment

The dataset contains 32,769,490 recorded rows. The submission repartitions by PlateValue, sorts within partitions by event_ts, and writes Parquet. This is a Spark data-layout optimization, rather than a classic B-tree index. The distinction matters because it is a documented deviation from the assignment's indexing method. The report compares a plate-filter operation on CSV at 63.78 seconds with the prepared Parquet path at 7.50 seconds. Their quotient is approximately 8.50; one other recorded timing comparison gives approximately 8.42, which must be labeled with its own observations. Differences include encoding, columnar scans, cached/warm execution, and layout, so the result cannot be attributed solely to an index. Parquet statistics and predicate pushdown can help skip data, but a hash repartition does not by itself guarantee direct file routing for every plate predicate. The actual plan and scan metrics should establish that mechanism. Filters on unrelated columns and broad aggregates can still require extensive reads. Even then, column pruning/compression can help, so claiming that every non-key query receives no benefit is too absolute. The retained source output shows the measured comparison without treating it as a universal speedup.

```python
import time
from pyspark.sql import SparkSession
from pyspark.sql.functions import col


spark = SparkSession.builder \
    .appName("Optimized_Index_Builder") \
    .master("local[*]") \
    .config("spark.sql.shuffle.partitions", "100") \
    .config("spark.shuffle.compress", "true") \
    .config("spark.shuffle.spill.compress", "true") \
    .config("spark.local.dir", "./spark_temp") \
    .getOrCreate()


INPUT_FILE = "plates_all_anonymized.csv"
INDEXED_OUTPUT_DIR = "indexed_data.parquet"
SAMPLE_PLATE_TO_TEST = 151506153

df_raw = spark.read.csv(INPUT_FILE, header=True, inferSchema=True, encoding="utf-8")

columns_to_keep = [
    "CompanyCode", "DeviceID", "Lane", "MasterPlateValue",
    "PassedTime", "ReceivedTime", "Speed", "VehicleType",
    "road_code", "road_name", "state_name", "state_code",
    "PlateValue", "event_ts"
]
df_pruned = df_raw.select(columns_to_keep)


df_clean = df_pruned.dropna(subset=["PlateValue", "event_ts", "DeviceID"])
```

Figure 1: Implementation and output: Data preparation and access experiment.

```python
print("--- Measuring execution BEFORE applying Index (Raw Data) ---")
start_time_unoptimized = time.time()
result_unoptimized = df_clean.filter(
    (col("PlateValue") == SAMPLE_PLATE_TO_TEST) &
    (col("event_ts").isNotNull())
).count()
end_time_unoptimized = time.time()
time_unoptimized = end_time_unoptimized - start_time_unoptimized
print(f"Time taken without index: {time_unoptimized:.4f} seconds (Found: {result_unoptimized} records)")

print("\n--- Creating Optimized Structure (B-Tree equivalent) ---")

df_optimized = df_clean.repartition(100, "PlateValue").sortWithinPartitions("PlateValue", "event_ts")

df_optimized.write.mode("overwrite").parquet(INDEXED_OUTPUT_DIR)
print(f"Optimized data saved to: {INDEXED_OUTPUT_DIR}")

print("\n--- Measuring execution AFTER applying Index ---")

df_indexed = spark.read.parquet(INDEXED_OUTPUT_DIR)

start_time_optimized = time.time()
result_optimized = df_indexed.filter(
    (col("PlateValue") == SAMPLE_PLATE_TO_TEST) &
    (col("event_ts").isNotNull())
).count()
end_time_optimized = time.time()
time_optimized = end_time_optimized - start_time_optimized
print(f"Time taken WITH index: {time_optimized:.4f} seconds (Found: {result_optimized} records)")

print("\n--- Summary ---")
print(f"Speedup Factor: {time_unoptimized / time_optimized:.2f}x")
```

Figure 2: Implementation and output: Data preparation and access experiment.

```
--- Measuring execution BEFORE applying Index (Raw Data) ---

Time taken without index: 63.7888 seconds (Found: 10 records)

--- Creating Optimized Structure (B-Tree equivalent) ---

Optimized data saved to: indexed_data.parquet

--- Measuring execution AFTER applying Index ---
[Stage 9:=============================================================>     (12 + 1) / 13]
Time taken WITH index: 7.5005 seconds (Found: 10 records)

--- Summary ---
Speedup Factor: 8.50x
```

Figure 3: Diagram or recorded result: Data preparation and access experiment.

2 Market baskets and time-window sensitivity

Each basket identifies a vehicle and a fixed event-time window. Items combine contractor and camera as CompanyCode_DeviceID. Sets collapse repeated sightings of the same item; a three-item basket means at least three distinct items, not merely three rows. The report compares 30-minute and one-hour windows. The printed narrative describes roughly 28.6 million and 26.4 million initial baskets, with retained triple-capable fractions approximately 1.75 percent (about half a million) and 4.15 percent (about 1.09 million), respectively. The one-hour choice retains more co-occurrences in this observation and was selected for subsequent mining. It is not proved universally optimal, and fixed-window boundaries can split a real trip regardless of duration. The small plate-like value 5 appears in unusually large geographically scattered baskets. It may represent a default/unreadable identifier or another data-quality issue, but its meaning requires the dataset's encoding contract. Retaining such shared placeholders can create artificial high-support patterns, so the cleaning rule and effect on results must be explicit.

```
print("\n--- Sample of Final Baskets (1 Hour Window) ---")
final_baskets.select("PlateValue", "TimeWindow.start", "TimeWindow.end", "items").show(5, truncate=False)
```

Figure 4: Implementation and output: Market baskets and time-window sensitivity.

```
from pyspark.sql.functions import col, window, concat_ws, collect_set, size


df_indexed = spark.read.parquet("indexed_data.parquet")

df_items = df_indexed.withColumn("Item", concat_ws("_", col("CompanyCode"), col("DeviceID")))

def create_baskets_and_evaluate(df, delta_t):
    print(f"\n--- Processing Baskets with \u0394t = {delta_t} ---")


    baskets_raw = df.groupBy(
        col("PlateValue"),
        window(col("event_ts"), delta_t).alias("TimeWindow")
    ).agg(

        collect_set("Item").alias("items")
    )


    baskets_valid = baskets_raw.filter(size(col("items")) >= 3)


    total_raw = baskets_raw.count()
    total_valid = baskets_valid.count()

    print(f"Total raw baskets created: {total_raw}")
    print(f"Total VALID baskets (>= 3 items): {total_valid}")
    print(f"Retention rate: {(total_valid/total_raw)*100:.2f}%")

    return baskets_valid


baskets_30m = create_baskets_and_evaluate(df_items, "30 minutes")
baskets_1h = create_baskets_and_evaluate(df_items, "1 hour")

final_baskets = baskets_1h
final_baskets.cache()
```

Figure 5: Implementation and output: Market baskets and time-window sensitivity.

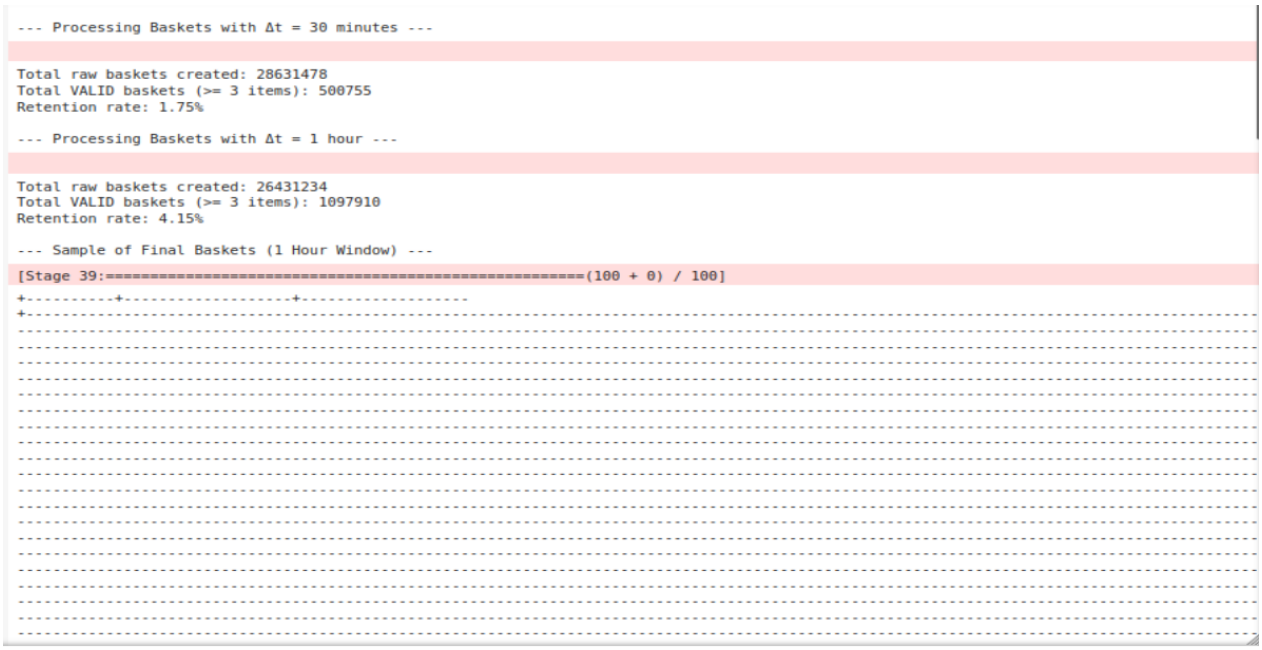


Figure 6: Diagram or recorded result: Market baskets and time-window sensitivity.



Figure 7: Diagram or recorded result: Market baskets and time-window sensitivity.

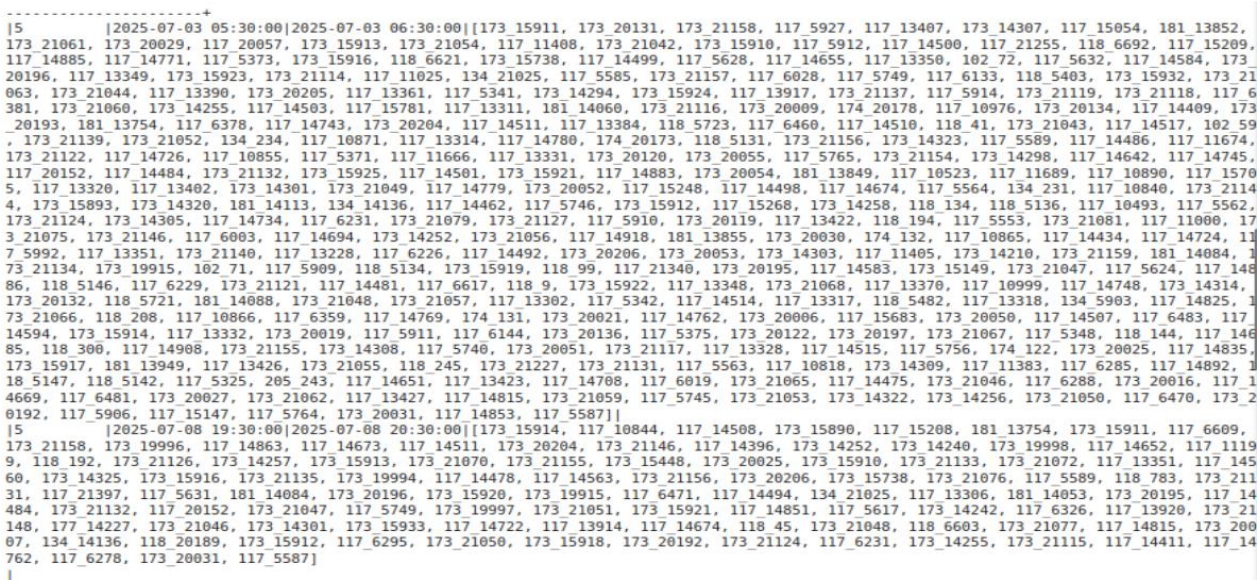


Figure 8: Diagram or recorded result: Market baskets and time-window sensitivity.

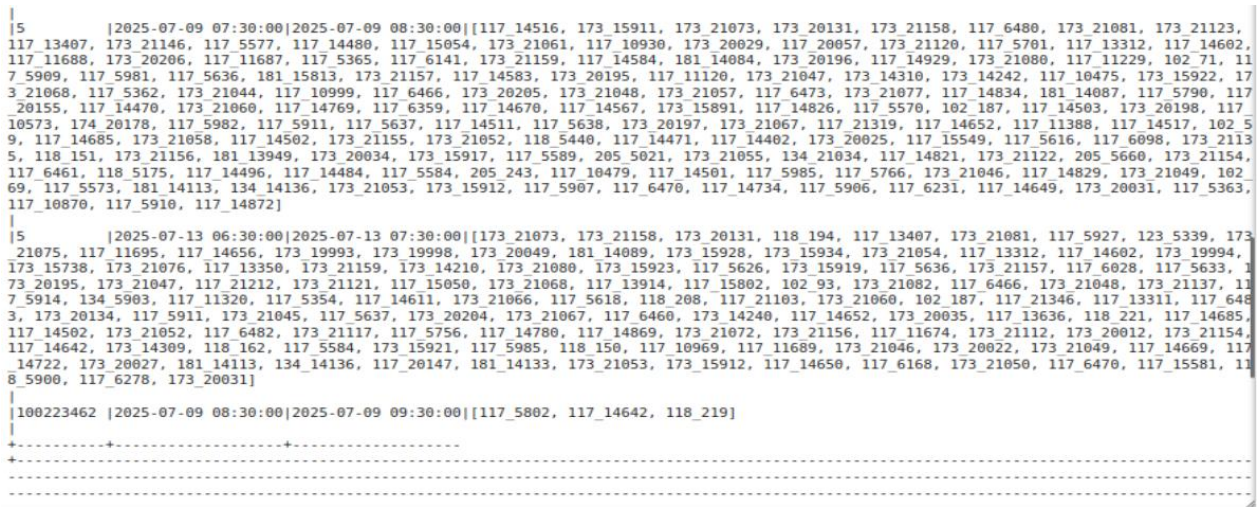


Figure 9: Diagram or recorded result: Market baskets and time-window sensitivity.

3 Frequent triples and algorithm choice

The submission uses Spark FP-Growth with minSupport=0.001, equivalent to 0.1 percent of the supplied baskets. The assignment requested Apriori or PCY; FP-Growth is a methodological substitution. Explain it as the actual submitted algorithm, rather than relabeling it to meet the requirement. Support counts baskets. The same vehicle can contribute several baskets, so a threshold near a thousand supporting baskets is not necessarily a thousand distinct vehicles. A frequent unordered triple does not prove A-to-B-to-C order, physical adjacency, or a mandatory corridor. The report observes repeated contractor codes such as 123 and 117 within high-support triples, suggesting a pattern worth checking against camera placement/contract data. It does not establish exact operator territory from co-occurrence alone.

```
from pyspark.ml.fpm import FPGrowth
from pyspark.sql.functions import col, size

print("--- Step 1: Filtering Noise (Data Cleaning) ---")

clean_baskets = final_baskets.filter(
    (col("PlateValue") > 1000) &
    (size(col("items")) <= 15)
)
clean_baskets.cache()

total_clean = clean_baskets.count()
print(f"Total clean baskets ready for mining: {total_clean}")

print(f"\n--- Step 2: Running FPGrowth Algorithm ---")

MIN_SUPPORT = 0.001
MIN_CONFIDENCE = 0.1

fpGrowth = FPGrowth(itemsCol="items", minSupport=MIN_SUPPORT, minConfidence=MIN_CONFIDENCE)

model = fpGrowth.fit(clean_baskets)

print("\n--- Step 3: Extracting 3-Itemsets ---")
frequent_itemsets = model.freqItemsets

itemsets_size_3 = frequent_itemsets.filter(size(col("items")) == 3).sort(col("freq").desc())

print(f"Top 10 Frequent 3-Itemsets (minSupport={MIN_SUPPORT}):")
itemsets_size_3.show(10, truncate=False)
```

Figure 10: Implementation and output: Frequent triples and algorithm choice.

```
--- Step 1: Filtering Noise (Data Cleaning) ---

Total clean baskets ready for mining: 1097651

--- Step 2: Running FPGrowth Algorithm ---
26/02/21 23:02:01 WARN FPGrowth: Input data is not cached.

--- Step 3: Extracting 3-Itemsets ---
Top 10 Frequent 3-Itemsets (minSupport=0.001):
[Stage 60:=============================================================> (98 + 2) / 100]
+---------------------------------+----+
|items                            |freq|
+---------------------------------+----+
|[123_5337, 123_5336, 123_5335]   |4226|
|[117_14551, 117_14552, 117_14568]|3811|
|[117_5312, 117_5316, 117_5315]   |3737|
|[117_5371, 117_5372, 117_5373]   |3694|
|[117_14547, 117_14549, 117_14548]|3624|
|[117_14547, 117_14553, 117_14548]|3604|
|[117_14549, 117_14553, 117_14548]|3540|
|[117_14567, 117_14549, 117_14548]|3479|
|[117_14552, 117_14568, 117_14553]|3431|
|[117_14559, 117_14554, 117_14560]|3301|
+---------------------------------+----+
only showing top 10 rows
```

Figure 11: Diagram or recorded result: Frequent triples and algorithm choice.

```python
import pyspark.sql.functions as F
from pyspark.sql.types import ArrayType, StringType

print("--- Step 1: Broadcasting Top Frequent Patterns ---")
top_patterns = [row['items'] for row in itemsets_size_3.limit(50).collect()]
top_patterns_b = spark.sparkContext.broadcast(top_patterns)

print("--- Step 2: Defining UDF to Calculate Misses ---")

def find_misses(basket_items):
    basket_set = set(basket_items)
    results = {}

    for pattern in top_patterns_b.value:
        pattern_set = set(pattern)
        intersect = pattern_set.intersection(basket_set)


        if len(intersect) == 3:
            for item in pattern_set:
                if item not in results: results[item] = [0, 0]
                results[item][1] += 1


        elif len(intersect) == 2:
            missed_item = (pattern_set - intersect).pop()
            if missed_item not in results: results[missed_item] = [0, 0]
            results[missed_item][0] += 1
            results[missed_item][1] += 1

            for item in intersect:
                if item not in results: results[item] = [0, 0]
                results[item][1] += 1


    return [f"{k}|{v[0]}|{v[1]}" for k, v in results.items()]

miss_udf = F.udf(find_misses, ArrayType(StringType()))
```

Figure 12: Implementation and output: Frequent triples and algorithm choice.

```
print("--- Step 3: Applying Logic and Aggregating Results ---")

df_eval = clean_baskets.withColumn("eval_results", miss_udf(F.col("items")))

df_exploded = df_eval.select(F.explode("eval_results").alias("eval_str"))
df_split = df_exploded.select(
    F.split("eval_str", "\|").getItem(0).alias("Item"),
    F.split("eval_str", "\|").getItem(1).cast("int").alias("MissCount"),
    F.split("eval_str", "\|").getItem(2).cast("int").alias("ExpectedCount")
)


camera_miss_rates = df_split.groupBy("Item").agg(
    F.sum("MissCount").alias("TotalMiss"),
    F.sum("ExpectedCount").alias("TotalExpected")
).withColumn("MissRate", F.round((F.col("TotalMiss") / F.col("TotalExpected")) * 100, 2))


camera_final = camera_miss_rates.withColumn("CompanyCode", F.split("Item", "_").getItem(0)) \
                                .withColumn("DeviceID", F.split("Item", "_").getItem(1))

# Aggregate results by company       (Company)
company_miss_rates = camera_final.groupBy("CompanyCode").agg(
    F.sum("TotalMiss").alias("CompanyTotalMiss"),
    F.sum("TotalExpected").alias("CompanyTotalExpected")
).withColumn("CompanyMissRate", F.round((F.col("CompanyTotalMiss") / F.col("CompanyTotalExpected")) * 100, 2))

print("\n--- Top 10 Cameras with Highest Miss Rate (Minimum 100 expected passes) ---")

camera_final.filter(F.col("TotalExpected") > 100).orderBy(F.col("MissRate").desc()).show(10, truncate=False)

print("\n--- Company Performance (Ranked by Miss Rate) ---")
company_miss_rates.filter(F.col("CompanyTotalExpected") > 500).orderBy(F.col("CompanyMissRate").desc()).show(10, truncate=False)
```

Figure 13: Implementation and output: Frequent triples and algorithm choice.

4 Camera and contractor miss heuristics

The method broadcasts the top 50 frequent triples and examines baskets containing some expected items while lacking another. It interprets those partial sets as suspected missing registrations. The denominator is exposure inferred from patterns, not independently observed vehicles passing every camera. The narrative identifies camera 11238 of contractor 117 with a reported rate around 37.88 percent. Company-level examples include contractor 181 around 24.45 percent, 117 around 23.59 percent, and a comparison around 20.77 percent for 173 versus 23.87 percent for 123. Large exposure makes a moderate rate operationally consequential; the report mentions contractor 117 with over 1.5 million expected instances and more than 356,000 inferred misses. These are pattern-based flags. Alternate routes, missing observations elsewhere, window splitting, identifiers, and repeated exposure across triples can change rates. A partial basket does not prove a dirty lens, camera angle error, or ANPR failure. Operator aggregation should use explicit exposure weighting rather than an unweighted average of camera rates.

```
--- Step 1: Broadcasting Top Frequent Patterns ---

--- Step 2: Defining UDF to Calculate Misses ---
--- Step 3: Applying Logic and Aggregating Results ---

--- Top 10 Cameras with Highest Miss Rate (Minimum 100 expected passes) ---

+---------+---------+-------------+--------+-----------+--------+
|Item     |TotalMiss|TotalExpected|MissRate|CompanyCode|DeviceID|
+---------+---------+-------------+--------+-----------+--------+
|117_11238|3637     |9602         |37.88   |117        |11238   |
|117_5312 |4641     |14511        |31.98   |117        |5312    |
|117_14563|2929     |9338         |31.37   |117        |14563   |
|123_5335 |4793     |15648        |30.63   |123        |5335    |
|123_5597 |2605     |9220         |28.25   |123        |5597    |
|117_14556|5054     |17988        |28.1    |117        |14556   |
|117_14567|16318    |58249        |28.01   |117        |14567   |
|123_5590 |2521     |9220         |27.34   |123        |5590    |
|117_5371 |3248     |11974        |27.13   |117        |5371    |
|117_14562|4803     |18093        |26.55   |117        |14562   |
+---------+---------+-------------+--------+-----------+--------+
only showing top 10 rows


--- Company Performance (Ranked by Miss Rate) ---
[Stage 68:=============================================================>(99 + 1) / 100]
+-----------+----------------+--------------------+---------------+
|CompanyCode|CompanyTotalMiss|CompanyTotalExpected|CompanyMissRate|
+-----------+----------------+--------------------+---------------+
|181        |2252            |9212                |24.45          |
|123        |17810           |74604               |23.87          |
|117        |356183          |1509875             |23.59          |
|173        |2749            |13238               |20.77          |
+-----------+----------------+--------------------+---------------+
```

Figure 14: Diagram or recorded result: Camera and contractor miss heuristics.

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


data = {
    'CompanyCode': ['181', '123', '117', '173'],
    'CompanyName': [
        'FardIran (Setadi)',          # 181
        'Basir Pardazesh (Setadi)',   # 123
        'Rahkam (Ostani)',            # 117
        'Basir Pardazesh (Ostani)'    # 173
    ],
    'MissRate': [24.45, 23.87, 23.59, 20.77],
    'TotalExpected': [9212, 74604, 1509875, 13238]
}

df_plot = pd.DataFrame(data)
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 6))
ax = sns.barplot(x='MissRate', y='CompanyName', data=df_plot, palette='Reds_r')


for p in ax.patches:
    width = p.get_width()
    plt.text(width + 0.3, p.get_y() + p.get_height()/2. + 0.1,
             f'{width:.2f}%', ha="center", fontsize=12, fontweight='bold')

plt.title('Miss Rate Comparison by Company Contractors', fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Miss Rate (%) - Lower is Better', fontsize=14)
plt.ylabel('Company Name', fontsize=14)
plt.xlim(0, 30)

plt.tight_layout()
plt.show()
```

Figure 15: Implementation and output: Camera and contractor miss heuristics.

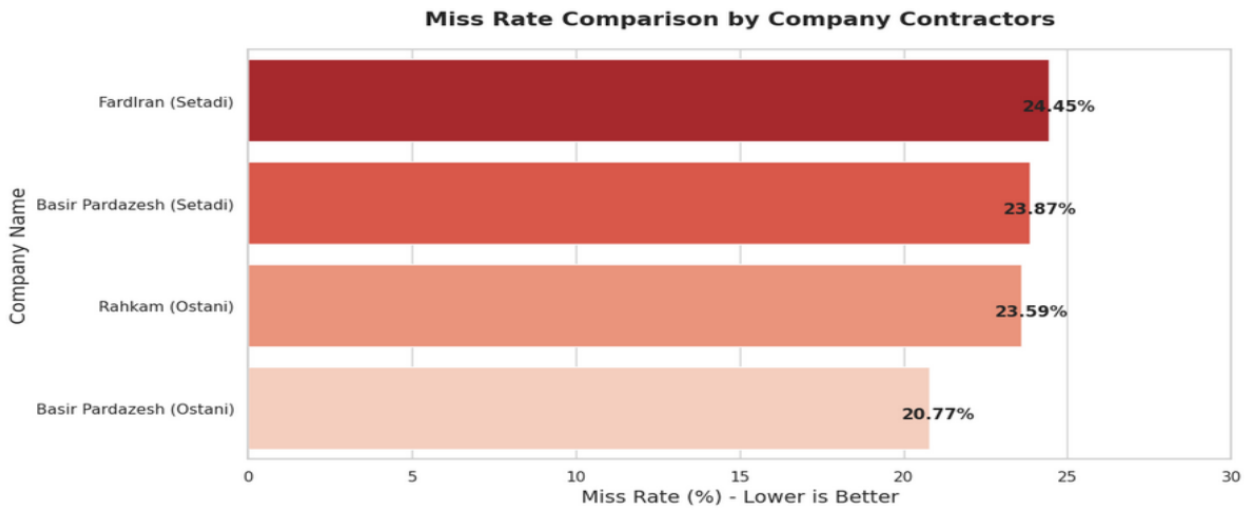


Figure 16: Diagram or recorded result: Camera and contractor miss heuristics.

5 MinHash/LSH route-set similarity

Represent each vehicle by a set of road_code values and use MinHash/LSH to produce candidates, with numHashTables=5 in the submitted Spark setup. Compute exact Jaccard on candidate sets. The report restricts execution to a random sample because of local resource limits and skew; therefore this is not a full-network all-pairs analysis. Pairs with Jaccard one have identical observed road sets, such as the displayed set containing 1338,1339,1012. They need not have the same order, direction, timestamp, or lane. Spark MinHashLSH's table parameter is not automatically a separately implemented textbook bands/rows design; document the actual API and candidate settings. Low route-set similarity can suggest that two vehicles have different observed coverage, but does not prove that a skipped camera is healthy. Perfect set similarity likewise does not prove an identical physical trip or eliminate all route confounders. LSH is approximate candidate generation and can omit similar pairs; exact Jaccard verifies only candidates considered.

```
import pyspark.sql.functions as F
from pyspark.ml.feature import CountVectorizer, MinHashLSH

print("--- Step 1: Extracting Route Sets per Vehicle ---")

df_indexed = spark.read.parquet("indexed_data.parquet")

vehicle_routes = df_indexed.filter(
    F.col("road_code").isNotNull() &
    (F.col("PlateValue") > 1000)
).groupBy("PlateValue").agg(
    F.collect_set(F.col("road_code").cast("string")).alias("routes")
).filter(F.size(F.col("routes")) >= 3)


# sampled_vehicles = vehicle_routes.sample(fraction=0.01, seed=42)

sampled_vehicles = vehicle_routes.limit(2000)
sampled_vehicles.cache()

total_vehicles = sampled_vehicles.count()
print(f"Total vehicles selected for LSH analysis: {total_vehicles}")

print("\n--- Step 2: Vectorization (Converting sets to ML Vectors) ---")

cv = CountVectorizer(inputCol="routes", outputCol="features", minDF=1.0)
cv_model = cv.fit(sampled_vehicles)
vectorized_routes = cv_model.transform(sampled_vehicles)

print("\n--- Step 3: Applying MinHash and LSH ---")

mh = MinHashLSH(inputCol="features", outputCol="hashes", numHashTables=5)
lsh_model = mh.fit(vectorized_routes)

print("\n--- Step 4: Finding Candidate Pairs and Calculating Jaccard ---")
```

Figure 17: Implementation and output: MinHash/LSH route-set similarity.

```python
JACCARD_DISTANCE_THRESHOLD = 0.3

similar_pairs = lsh_model.approxSimilarityJoin(
    vectorized_routes, vectorized_routes, JACCARD_DISTANCE_THRESHOLD, distCol="JaccardDistance"
).filter(
    F.col("datasetA.PlateValue") < F.col("datasetB.PlateValue")
)

print("\n--- Top Similar Vehicle Pairs ---")

results_df = similar_pairs.select(
    F.col("datasetA.PlateValue").alias("Vehicle_1"),
    F.col("datasetB.PlateValue").alias("Vehicle_2"),
    F.col("datasetA.routes").alias("Routes_1"),
    F.col("datasetB.routes").alias("Routes_2"),
    F.round((1 - F.col("JaccardDistance")), 3).alias("JaccardSimilarity")
).orderBy(F.col("JaccardSimilarity").desc())

results_df.show(10, truncate=False)
```

Figure 18: Implementation and output: MinHash/LSH route-set similarity.

```
--- Step 1: Extracting Route Sets per Vehicle ---

Total vehicles selected for LSH analysis: 2000

--- Step 2: Vectorization (Converting sets to ML Vectors) ---

--- Step 3: Applying MinHash and LSH ---

--- Step 4: Finding Candidate Pairs and Calculating Jaccard ---

--- Top Similar Vehicle Pairs ---
[Stage 32:>                                                         (0 + 1) / 1]
+---------+---------+--------------------------+--------------------------+-----------------+
|Vehicle_1|Vehicle_2|Routes_1                  |Routes_2                  |JaccardSimilarity|
+---------+---------+--------------------------+--------------------------+-----------------+
|1543084  |8785922  |[1338.0, 1339.0, 1012.0]  |[1338.0, 1339.0, 1012.0]  |1.0              |
|100151406|112867075|[2127.0, 782.0, 1738.0]   |[2127.0, 782.0, 1738.0]   |1.0              |
|100231246|131985638|[807.0, 787.0, 809.0, 808.0]|[807.0, 787.0, 809.0, 808.0]|1.0            |
|100624433|113096822|[2034.0, 2019.0, 2014.0]  |[2034.0, 2019.0, 2014.0]  |1.0              |
|100666783|153154142|[1010.0, 346.0, 347.0]    |[1010.0, 346.0, 347.0]    |1.0              |
|101514247|160235647|[1774.0, 2033.0, 1843.0]  |[1774.0, 2033.0, 1843.0]  |1.0              |
|101588176|103053774|[2033.0, 2021.0, 2015.0]  |[2033.0, 2021.0, 2015.0]  |1.0              |
|101778233|131946425|[642.0, 393.0, 687.0]     |[642.0, 393.0, 687.0]     |1.0              |
|102181564|112911310|[648.0, 185.0, 647.0, 186.0]|[648.0, 185.0, 647.0, 186.0]|1.0            |
|102410774|161058226|[2021.0, 1200.0, 1199.0]  |[2021.0, 1200.0, 1199.0]  |1.0              |
+---------+---------+--------------------------+--------------------------+-----------------+
only showing top 10 rows
```

Figure 19: Diagram or recorded result: MinHash/LSH route-set similarity.

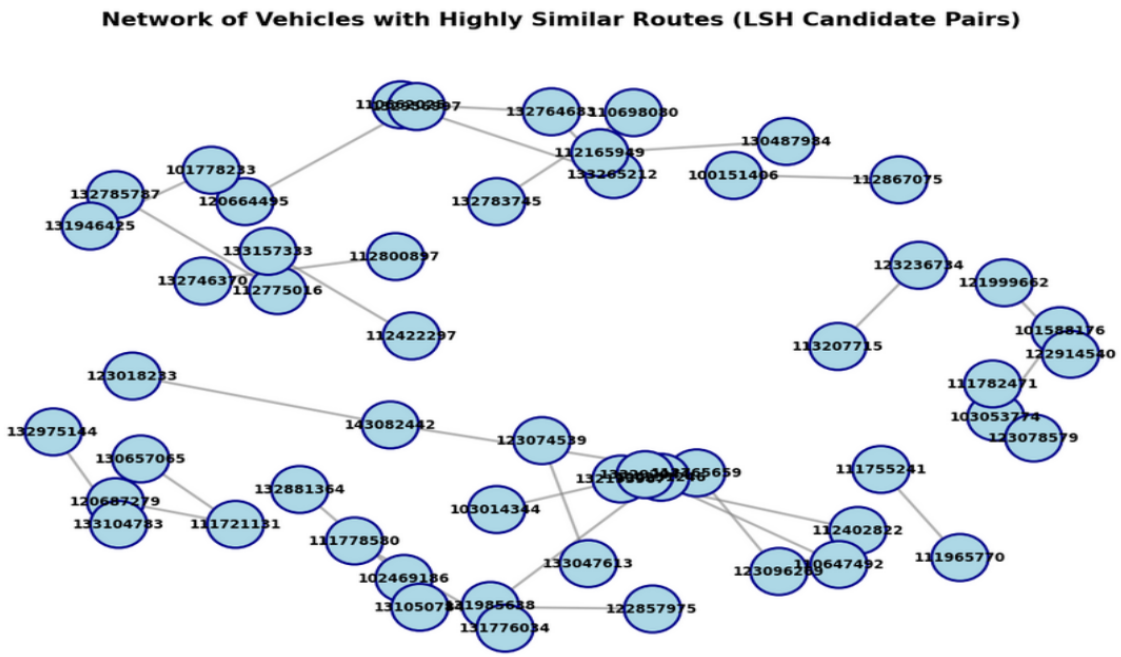


Figure 20: Diagram or recorded result: MinHash/LSH route-set similarity.

6 Engineering interpretation and preserved results

The project joins data layout, basket mining, and route similarity into a framework for proposing quality checks. Its useful contribution is ranking suspicious patterns with clear evidence, not a validated hardware-failure classifier. Reproduction uses the notebook and dataset schema. Performance interpretation is limited to the recorded sample and execution environment.